

NBA Player Props Model – Technical Documentation

Author: Joey Shackelford

Last Updated: November 2025

Overview

I built this model to generate complete probability distributions for NBA player prop markets. The main difference from standard approaches is that instead of just outputting "LeBron has a 52% chance to go over 25.5 points", it generates the full probability mass function — $P(X=20)$, $P(X=21)$, $P(X=22)$, etc. for all possible values.

This matters because it lets you price any line without retraining. Want odds on LeBron at 19.5, 20.5, 21.5, all the way to 35.5? One model run gives you all of them.

Architecture

The model has six layers:

Layer 1: Base Models

I'm using five different algorithms trained on the same features:

- XGBoost (my main workhorse — handles non-linear interactions well)
- LightGBM (fast, good with high-dimensional data)
- CatBoost (built for categorical features like opponent and home/away)
- Neural Network (4 hidden layers with batch normalization)
- Random Forest (provides a stable baseline)

Why five? Because each has different biases. XGBoost likes additive interactions, neural nets can model complex patterns but need more data, random forests are stable but sometimes too simple. Combining them reduces the risk that one algorithm's quirk becomes your model's blind spot.

Layer 2: Player-Specific Models

For players with enough data (30+ games), I train individualized models. Research shows this improves accuracy by about 2% for high-volume players who have unique styles or usage patterns.

The logic:

- Less than 30 games → Use global model
- 30–100 games → Train lightweight XGBoost only (prevent overfitting)
- 100+ games → Train full ensemble

Layer 3: Meta-Learner (Stacking)

A Ridge regression model sits on top of the base models and learns how to weight their predictions. I use 5-fold time-series cross-validation to prevent leakage.

Ridge is nice here because it's linear (interpretable) and the L2 penalty handles when base models are correlated. Typical weights it learns:

- XGBoost: 0.28
- LightGBM: 0.24
- CatBoost: 0.22
- Neural Net: 0.16
- Random Forest: 0.10

Layer 4: Distribution Fitting

This is where it gets interesting. I take the point prediction and fit it to a Negative Binomial distribution using the mean from the model and variance from cross-validation residuals.

For counting stats (points, rebounds, assists), Negative Binomial works well because it handles overdispersion — when variance > mean, which is common in sports.

Then I generate $P(X = n)$ for every value from 0 to max_value (usually 60 for points). This is the complete PMF.

Why this matters:

- You can price any line (not just market lines)
- You can build same-game parlays with proper correlation
- You can hedge dynamically as lines move
- Tail risk for extreme outcomes is properly quantified

Layer 5: Calibration

Raw probabilities from the distribution are often miscalibrated. Like, the model says 60% but it actually happens 55% of the time.

I use isotonic regression on held-out validation data (20% of games, chronologically separated) to map predicted probabilities to observed frequencies. This is critical for margin-building — if your probabilities are off, your lines will be systematically exploitable.

I track Brier score (targeting < 0.20) to measure calibration quality.

Layer 6: Market Intelligence

- Line movement: If sharp books like Pinnacle move a line significantly, I reduce my edge. They probably know something I don't.
- Steam moves: Synchronized movement across books = informed money. Lower confidence.
- Closing line value: I track how my opening lines compare to market close. Positive CLV means I have edge.

The adjustment multiplier:

- High confidence (1.0): No sharp action, stable lines
- Medium (0.7): Some movement, uncertain

- Low (0.4): Clear sharp action detected
- None (0.0): Line pulled or major reverse movement

Margin-Building

This is where most models get it wrong. The standard approach is: calculate fair odds, then add juice. Problem is, that's crude and doesn't maintain proper probability structure.

I build margin in probability space first, then convert to odds. Four methods implemented:

- Method 1: Power (Shin, 1991) — Apply a power transformation ($k < 1$) to both sides, normalize to sum to 1 + margin. Most efficient method — minimizes distortion.
- Method 2: Multiplicative (Favorite-Longshot Bias) — Applies less margin to favorites, more to longshots. Reflects how real markets behave.
- Method 3: Additive — Simple proportional allocation. Easy but less sophisticated.
- Method 4: Odds-Ratio (Logarithmic) — Works in log-odds space. Better tail behavior for extreme probabilities.

In practice, I default to the power method. Use multiplicative when favorite-longshot bias is pronounced.

Features

Temporal:

- Rolling averages (last 3, 5, 7, 10, 15 games)
- Exponentially weighted averages (half-life = 5 games)
- Trend indicators (recent vs. previous performance)
- Volatility (rolling standard deviation)

Matchup:

- Opponent defensive rating (last 10 games)
- Opponent pace
- Historical performance vs. this opponent
- Days rest

Situational:

- Home/away
- Back-to-back flag
- Minutes restrictions (injury, load management)
- Usage rate

Derived:

- Minutes × usage rate (volume proxy)

- Pace × minutes (opportunity indicator)
- Opponent defensive rating × usage rate (difficulty-adjusted volume)

Feature selection: Recursive elimination with 10-fold CV. Keep features with importance > 0.01 or strong domain justification.

Training Approach

Data Split:

Time-series split (no random shuffling):

- Training: 70% (earliest games)
- Validation: 15% (middle chronologically)
- Test: 15% (most recent games)

Cross-Validation:

Walk-forward with expanding window. Train on all data up to time t , validate on $t+1$ to $t+k$, repeat.

Hyperparameters:

Bayesian optimization via Optuna. 100 trials per model. Optimize for MAE on validation set.

Retraining:

Weekly during season. Daily for player-specific models when new data is available.

Performance

Point Prediction:

- MAE: 3.5–4.5 points
- RMSE: 5.0–6.5 points
- R^2 : 0.65–0.75

Probabilistic Calibration:

- Brier score: < 0.20
- Log loss: < 0.50
- Calibration curve: Slope ≈ 1.0 , intercept ≈ 0.0

Market Performance (backtested):

- Win rate: 58–60% (on opportunities with >3% edge)
- CLV: Positive (opening lines better than closing)
- ROI: 5–8% (with Kelly sizing)

Production Considerations

Latency:

- Single prediction: < 100ms
- Full slate (300+ props): < 30 seconds
- Model retraining: ~2 hours (weekly batch)

Scalability:

- 50 players × 6 prop types = 300 predictions per slate
- 82 games × 2 slates per day = ~50K predictions per season
- Horizontal scaling via player-level parallelization; model serving via FastAPI with load balancing; cached feature engineering

Monitoring:

- Prediction distribution drift
- Calibration degradation (weekly Brier score)
- Market performance (daily CLV tracking)
- Feature importance stability

Failure Modes:

- Missing data (injury reports not updated) → Fall back to prior game features
- Model uncertainty too high → Flag for manual review
- Market disagreement > 10% → Reduce position size

What's Missing (And I Know It)

- Real-time injury/lineup integration: needs a preprocessing pipeline that hits injury APIs and adjusts features in real-time.
- Market feedback loops: need access to sharp money movement and closing lines at scale (desk feed or scraping).
- Correlation modeling for SGPs: architecture supports it; copulas next on roadmap.
- In-game adjustments: would require tick-level data (foul trouble, minute restrictions, momentum).

How Easily Can This Be Replicated?

Pretty easily if you have the infrastructure. The margin-building methodology is published (Shin 1991), stacking ensembles aren't novel, and Negative Binomial fitting is standard.

What takes time:

- Proper calibration (need held-out data and multiple iterations)
- Feature engineering for edge cases (injuries, rest, matchups)
- Integration into a production trading system with risk controls

- Backtesting and validation at scale

With clean data access and a team, I'd estimate 2–3 weeks to get a production version running. Solo took me longer because I had to handle data collection, cleaning, and validation myself.

90-Day Roadmap

If I were deploying this in a production environment, here's what I'd prioritize:

Weeks 1–4: Deploy & Validate

- Integrate into production for NBA props
- Establish baseline metrics and monitoring dashboards
- Begin daily CLV tracking

Weeks 5–8: Correlation Modeling

- Implement copula-based correlation for SGPs
- Build correlation matrices (points–rebounds, points–assists, etc.)

Weeks 9–12: Real-Time Features

- Ingest live injury reports
- Build within-game adjustment logic
- Reduce latency from lineup announcement to line updates

Months 4–6: Scale to Other Sports

- Extend to NFL (same framework, different features)
- Then MLB, NHL

Months 6–12: Advanced Features

- Injury probability modeling
- Dynamic line adjustment via RL
- Proprietary edge identification

Why This Approach

- Output: Standard = $P(X > \text{line})$ for one line; This model = $P(X = n)$ for all n
- Margin: Standard = Add vig to odds; This model = Build in probability space (4 methods)
- Architecture: Standard = Single model; This model = 6-layer meta-ensemble
- Calibration: Standard = Assumed; This model = Isotonic regression on validation
- Uncertainty: Standard = Point estimate $\pm$ fixed error; This model = Full probability distribution

This isn't just a better model. It's a different way of thinking about the problem. Instead of "what's our line on LeBron O25.5", it's "give me the complete probability distribution for LeBron's points tonight." That's what enables dynamic pricing, SGP construction, and systematic edge identification.

Contact

Contact: Joseph Shackelford

Repository: <https://github.com/Risky-Scout/nba-player-props-model>

Last Updated: November 2025